

## 104. Maximum Depth of Binary Tree Easy

Acceptance: 77.8% | [LeetCode](#)

Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

### Example 1:

```
**Input:** root = [3,9,20,null,null,15,7]
**Output:** 3
```

### Example 2:

```
**Input:** root = [1,null,2]
**Output:** 2
```

### Constraints:

- The number of nodes in the tree is in the range `[0, 104]`.
- `-100 <= Node.val <= 100`

## Claude Opus Solution

## Maximum Depth of Binary Tree

### 1. Intuition

The key insight is **recursive self-similarity**: the depth of a tree is just **1 (for the current node) + the depth of its deeper subtree**.

Think of it this way: to know how tall a tree is, ask "How tall is my left branch? How tall is my right branch?" Whichever is taller, add 1 (for myself), and that's the answer. Each subtree is itself a smaller tree, so the **same logic applies recursively** until we hit empty nodes (depth 0).

This is a classic **Depth-First Search (DFS)** pattern on trees.

### 2. Approach (Pseudocode)

```
function maxDepth(node):
    # Base case: an empty node contributes 0 to depth
    if node is null:
        return 0

    # Recursively compute depth of each subtree
    leftDepth = maxDepth(node.left)
    rightDepth = maxDepth(node.right)

    # Current depth = deeper child + 1 (for current node)
    return max(leftDepth, rightDepth) + 1
```

### 3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        # BASE CASE:
        # If the node is None (empty tree or reached past a leaf),
        # it adds 0 levels of depth.
        if root is None:
            return 0

        # RECURSIVE CASE:
```

```
# Find the depth of the left subtree.
left_depth = self.maxDepth(root.left)

# Find the depth of the right subtree.
right_depth = self.maxDepth(root.right)

# The depth at the current node is the greater of the two
# subtree depths, plus 1 to count the current node itself.
return max(left_depth, right_depth) + 1
```

**Note:** The reference `my_solution.py` adds an explicit leaf check (`if root.left is None and root.right is None: return 1`). This is *redundant* — when both children are `None`, the recursion naturally returns `max(0, 0) + 1 = 1`. The cleaner version above avoids this unnecessary check.

## 4. Complexity Analysis

| Metric | Complexity | Explanation                                                                                                                                                         |
|--------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Time   | $O(n)$     | We visit every node exactly once, where $n$ is the number of nodes.                                                                                                 |
| Space  | $O(h)$     | The recursion call stack holds at most $h$ frames, where $h$ is the tree height. <b>Best/balanced case:</b> $O(\log n)$ . <b>Worst case (skewed tree):</b> $O(n)$ . |

## 5. Alternative Approaches

### A) Iterative DFS (using an explicit stack)

Avoids recursion limits by simulating the call stack manually:

```
def maxDepth(self, root: Optional[TreeNode]) -> int:
    if not root:
        return 0
    stack = [(root, 1)] # store (node, current_depth)
    max_depth = 0
    while stack:
        node, depth = stack.pop()
        if node:
            max_depth = max(max_depth, depth)
            stack.append((node.left, depth + 1))
            stack.append((node.right, depth + 1))
    return max_depth
```

- Time  $O(n)$ , Space  $O(h)$ .

### B) Iterative BFS (level-order traversal)

Count the number of **levels** by processing the tree one level at a time:

```
from collections import deque

def maxDepth(self, root: Optional[TreeNode]) -> int:
    if not root:
        return 0
    queue = deque([root])
    depth = 0
    while queue:
        depth += 1 # one more level processed
        for _ in range(len(queue)): # process all nodes in current level
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return depth
```

- Time  $O(n)$ , Space  $O(w)$ , where  $w$  is the maximum width of the tree.

## Summary

| Approach                                                                                        | Time   | Space  | When to Use                       |
|-------------------------------------------------------------------------------------------------|--------|--------|-----------------------------------|
| Recursive DFS  | $O(n)$ | $O(h)$ | Cleanest, default choice          |
| Iterative DFS                                                                                   | $O(n)$ | $O(h)$ | Avoid deep recursion limits       |
| Iterative BFS                                                                                   | $O(n)$ | $O(w)$ | When you need level-by-level info |

The **recursive DFS** is the recommended solution for its elegance and clarity.

## Codex Solution

### 1. Intuition

The maximum depth of a binary tree is the length of the longest path from the current node down to a leaf.

For any node:

```
maxDepth(node) = 1 + max(maxDepth(node.left), maxDepth(node.right))
```

Why?

- The current node contributes `1` to the depth.
- Then we choose the deeper subtree between the left and right child.
- If the node is `None`, its depth is `0`.

This is a classic **recursive DFS** pattern on binary trees.

### 2. Approach

Use recursion to compute the maximum depth of each subtree.

#### Pseudocode

```
function maxDepth(root):  
    if root is None:  
        return 0  
  
    leftDepth = maxDepth(root.left)  
    rightDepth = maxDepth(root.right)  
  
    return 1 + max(leftDepth, rightDepth)
```

### 3. Python Solution

```
from typing import Optional  
  
# Definition for a binary tree node.  
# LeetCode already provides this class, so usually you do not need to redefine it.  
class TreeNode:  
    def __init__(self, val=0, left=None, right=None):  
        self.val = val          # Value stored in the node  
        self.left = left        # Left child  
        self.right = right      # Right child  
  
class Solution:  
    def maxDepth(self, root: Optional[TreeNode]) -> int:  
        # Base case:  
        # An empty tree has depth 0.  
        if root is None:  
            return 0  
  
        # Recursively compute the maximum depth of the left subtree.  
        left_depth = self.maxDepth(root.left)  
  
        # Recursively compute the maximum depth of the right subtree.  
        right_depth = self.maxDepth(root.right)
```

```
# The depth of the current tree is:  
# 1 for the current node  
# plus the larger depth of its two subtrees.  
return 1 + max(left_depth, right_depth)
```

---

## 4. Complexity Analysis

Let  $n$  be the number of nodes in the binary tree.

**Time Complexity:**  $O(n)$

Each node is visited exactly once.

For every node, we do constant work:

- Check if it is `None`
- Recursively compute left and right depths
- Take the maximum

Therefore, total time complexity is:

```
O(n)
```

**Space Complexity:**  $O(h)$

The recursion stack stores one function call for each node on the current path.

Here,  $h$  is the height of the tree.

- Best case, balanced tree:  $O(\log n)$
- Worst case, completely skewed tree:  $O(n)$

So the space complexity is:

```
O(h)
```

---

## 5. Alternative Approaches

### Alternative 1: Iterative BFS

We can perform level-order traversal using a queue.

Each level we process increases the depth by `1`.

```
from collections import deque  
from typing import Optional  
  
class Solution:  
    def maxDepth(self, root: Optional[TreeNode]) -> int:  
        # Empty tree has depth 0.  
        if root is None:  
            return 0  
  
        queue = deque([root])  
        depth = 0  
  
        # Process the tree level by level.  
        while queue:  
            # Each loop iteration processes one full level.  
            level_size = len(queue)  
  
            for _ in range(level_size):  
                node = queue.popleft()  
  
                if node.left:  
                    queue.append(node.left)  
  
                if node.right:  
                    queue.append(node.right)  
  
            # After processing one level, increase depth.  
            depth += 1
```

```
return depth
```

## Complexity

- Time:  $O(n)$
  - Space:  $O(w)$ , where  $w$  is the maximum width of the tree
  - Worst case:  $O(n)$
- 

## Alternative 2: Iterative DFS

We can use a stack and store each node along with its current depth.

```
from typing import Optional

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        # Empty tree has depth 0.
        if root is None:
            return 0

        # Stack stores pairs: node and its depth.
        stack = [(root, 1)]
        max_depth = 0

        while stack:
            node, depth = stack.pop()

            # Update the maximum depth seen so far.
            max_depth = max(max_depth, depth)

            # Add children with depth + 1.
            if node.left:
                stack.append((node.left, depth + 1))

            if node.right:
                stack.append((node.right, depth + 1))

        return max_depth
```

## Complexity

- Time:  $O(n)$
  - Space:  $O(h)$  on average for DFS stack
  - Worst case:  $O(n)$
- 

The recursive DFS solution is the simplest and most common approach for this problem.